CP2 Report 2026 Assignment 2 (20264003, JuneyoungYoon)

All implementations are in `scripts/` and results are in `results/`.

1. Image Blending

1.1. Gaussian and Laplacian Pyramids

Implementation. The pyramids are built from a 5×5 Gaussian kernel (σ = 1) and a convolution written with NumPy. `Reduce` blurs the image and keeps every second row and column. `expand` inserts zeros between the samples, blurs with the same kernel and multiplies by 4. `gaussian_pyramid` applies `reduction` six times to obtain 7 levels. `laplacian_pyramid` stores L_k = G_k − expand(G_{k+1}) for k = 0…5 and keeps the coarsest Gaussian level as L_6. `collapse` reverses this from the coarsest level: I_k = L_k + expand(I_{k+1}). (scripts/Image_Blending/part1_1.py)

Details.

1. **Convolution.** `conv2d` pads the image with `np.pad(..., mode='symmetric')`, flips the kernel, and accumulates `kernel[i, j] × (shifted image)` over all kernel taps. Zero padding would darken the image borders at every level, so mirrored padding is used.
2. **Kernel.** `get_gaussian_kernel(size, sigma)` evaluates the 2D Gaussian on a size×size grid and normalizes its sum to 1, so blurring keeps the mean brightness.
3. **Factor 4 expands.** Zero insertion keeps one sample in four, so after blurring the image is about 1/4 as bright. Multiplying by 4 restores the brightness.
4. **Odd sizes.** `expand(image, shape)` takes the target shape of the finer level, so a level with an odd size can still be subtracted from its parent. For the 512×512 inputs every level size is even.
5. **Precision.** Images are converted to float64 in [0, 1] before building the pyramids. Laplacian levels are signed, so they are displayed as 0.5 + 0.5·L.

Output.

Image	Level sizes (7 levels)	max |error|	MSE
apple	512, 256, 128, 64, 32, 16, 8	2.22e-16	1.40e-36
orange	512, 256, 128, 64, 32, 16, 8	1.11e-16	1.29e-37

The reconstruction error is at the level of float64 rounding (about 1e-16). `collapse` uses exactly the same `expand` as `laplacian_pyramid`, so adding L_k back undoes each subtraction. The Laplacian level stores precisely what `expand` cannot recover, which makes the reconstruction exact for any kernel.

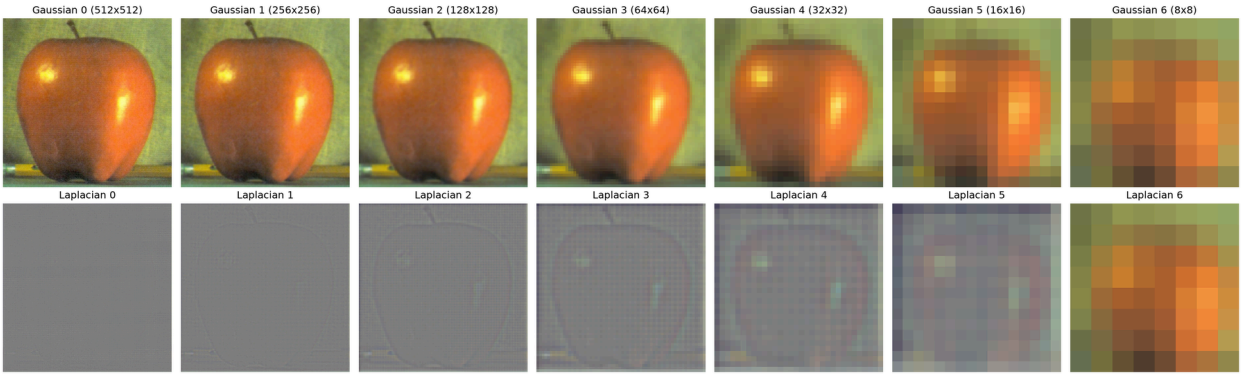

Figure 1. Gaussian pyramid (top) and Laplacian pyramid (bottom) of apple.png, 7 levels from 512×512 to 8×8.

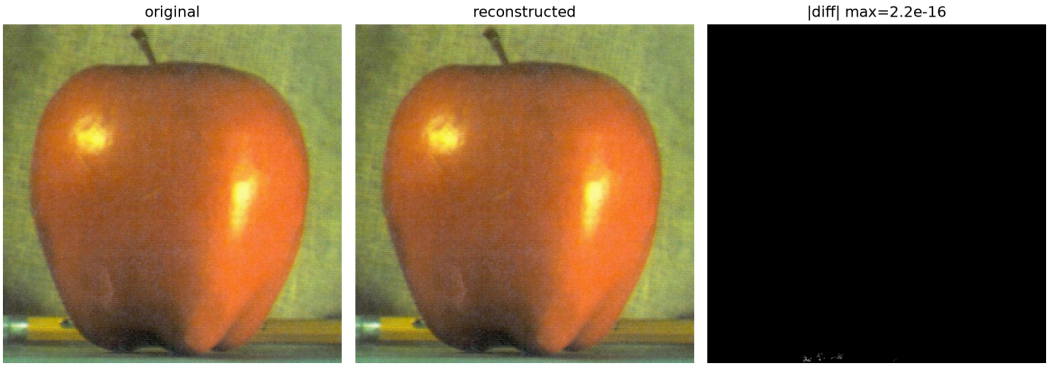

Figure 2. apple.png: original, image reconstructed by collapsing the Laplacian pyramid, and absolute difference.

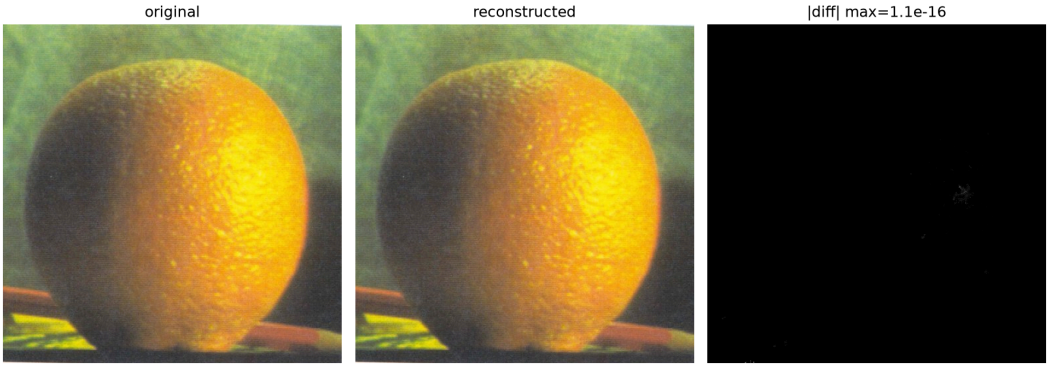

Figure 3. orange.png: original, reconstructed image, and absolute difference.

1.2. Multi-band Image Blending

Implementation. The Laplacian pyramids of apple (I_A) and orange (I_B) and the Gaussian pyramid of the mask are combined level by level and the result is collapsed. The mask is 1 on the left half (apple) and 0 on the right half (orange). For comparison, the images are also pasted directly as I_A·M + I_B·(1 − M). (scripts/Image_Blending/part1_2.py)

Details.

- `make_mask(shape, split=0.5)` builds a 3 channel binary mask. Its Gaussian pyramid is computed with the same `reduce` as the images, so m_k at a coarse level is a strongly blurred step.
- At level k one pixel covers 2^k pixels of the original image. A transition of a few pixels in m_k therefore spans roughly 2^k original pixels. Low frequencies are mixed over a wide band, and high frequencies over a narrow band. This is why the line disappears without the two textures overlapping.
- The two inputs must have the same size (both are 512×512).

Output.

Direct pasting leaves a sharp vertical line. The multi-band result changes naturally from the red apple to the orange, and the dimpled orange texture ends near the center line without visible doubling. The mean absolute difference between the two results is 0.0083 (in [0, 1]).

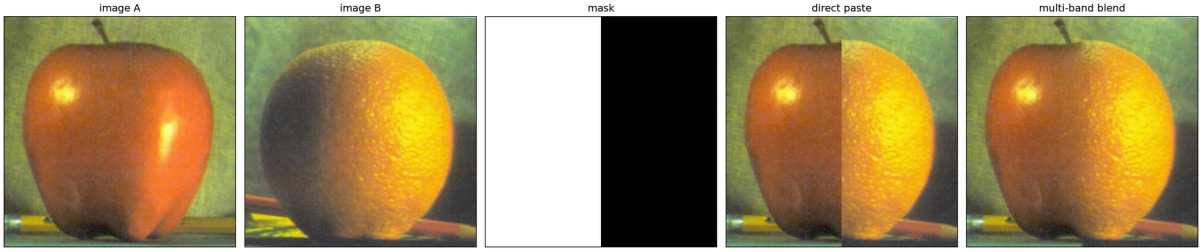

Figure 4. Inputs, mask, direct paste, and multi-band blend.

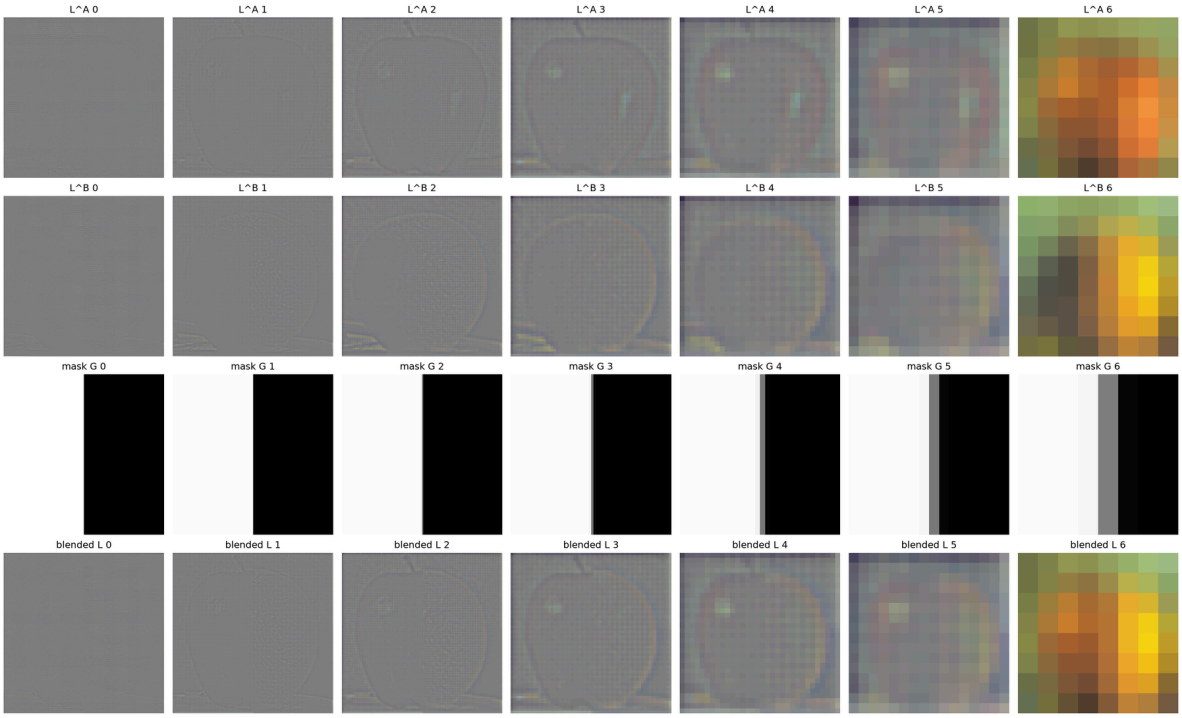

Figure 5. Laplacian pyramids of A and B, Gaussian pyramid of the mask, and the blended Laplacian pyramid.

Question. How does changing the number of pyramid levels affect the blending result? How does changing the Gaussian kernel size affect the blending result?

To answer this, the blend was repeated with 1, 3, 5 and 7 levels (5×5 kernel), and with 3×3, 5×5, 9×9 and 15×15 kernels (7 levels). The kernel σ grows with the size as σ = (size − 1)/4, so the kernel radius is 2σ and the default 5×5 keeps σ = 1.

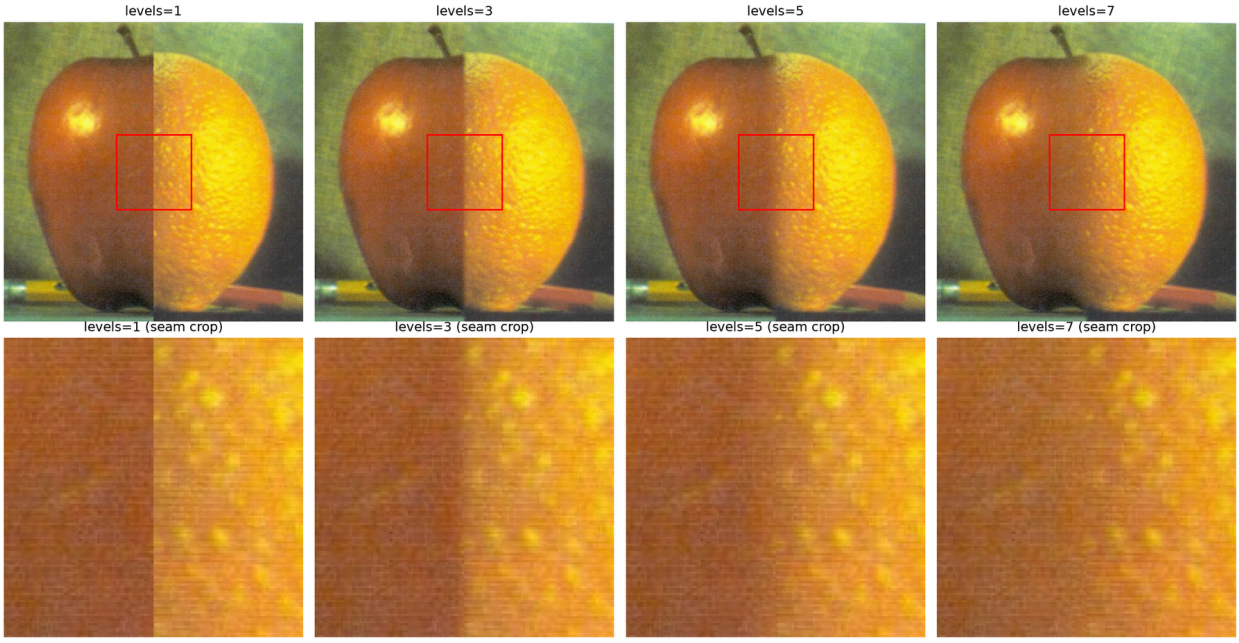

Figure 6. Blending with 1, 3, 5 and 7 pyramid levels (5×5 kernel). Bottom: seam crops.

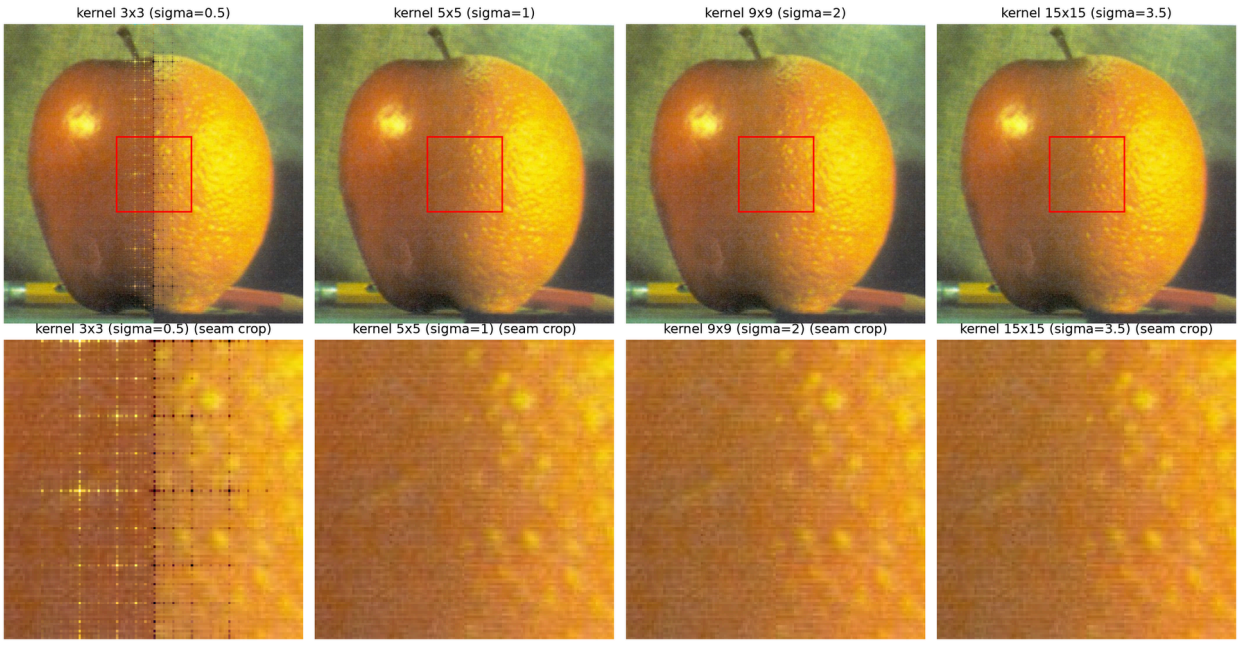

Figure 7. Blending with 3×3, 5×5, 9×9 and 15×15 kernels (7 levels). Bottom: seam crops.

Answer.
As the pyramid levels go up, the colors transition smoothly and naturally. At levels 1, 3, and 5, distinct color differences are clearly visible, while the blending lines still remain sharp. By the time it reaches level 7, the color differences largely fade away, resulting in a seamlessly blended image. And for kernel size, During pyramid blending, a 3×3 kernel ($\sigma = 0.5$) lacks sufficient aliasing suppression, causing grid-like artifacts near the seam lines. Therefore, using a kernel of 5×5 or larger ($\sigma \approx 1$) is optimal to effectively block aliasing while preventing unnecessary computational overhead.

2. Image Enhancement

Both parts use `palace.jpeg` (1500 × 2000, W × H). A photo of the palace courtyard I took.

Part 1. Detail Enhancement: Unsharp Masking

Implementation. Each RGB channel is processed with $I_{out} = I_{in} + \alpha(I_{in} - G_\sigma * I_{in})$, reusing `get_gaussian_kernel` and `conv2d` from Part 1.1. The single convolution(which was mentioned in class) version builds one kernel $K = (1 + \alpha)\delta - \alpha G_\sigma$, where δ is the identity kernel, and computes $I_{out} = K * I_{in}$. $\alpha \in \{1, 2, 3, 4\}$ is compared at $\sigma = 1.4$, and $\sigma \in \{0.5, 1, 2, 3\}$ at $\alpha = 1.5$.
(scripts/Image_Enhancement/part2_1.py)

Details.

- **Single kernel.** Because convolution is linear, $I + \alpha(I - G*I) = (1 + \alpha)I - \alpha G*I = ((1 + \alpha)\delta - \alpha G) * I$. The two versions differ by at most 6.0e-15 over all eight settings, i.e. only by floating-point rounding.
- **Kernel size.** `kernel_size(σ) = 2⌈3σ⌉ + 1` keeps the Gaussian up to 3σ (5×5 for $\sigma = 0.5$, 9×9 for $\sigma = 1.4$, 19×19 for $\sigma = 3$).
- **Flat regions.** The sum of K is $(1 + \alpha) - \alpha = 1$, so a region of constant intensity is unchanged. Only the neighborhood of edges and texture changes.
- **Range.** The output can leave [0, 1] near strong edges (overshoot). It is clipped to [0, 1] for display and saving.
- **Crops.** Detail is easier to judge at pixel level, so each comparison also shows a 128×128 crop taken 250 px above the image center, on the carved stucco.

Output.

Setting	Kernel	max |two-step − single kernel|
$\alpha = 1, 2, 3, 4$ ($\sigma = 1.4$)	9×9	2.66e-15, 3.00e-15, 5.22e-15, 6.00e-15
$\sigma = 0.5, 1.0, 2.0, 3.0$ ($\alpha = 1.5$)	5×5, 7×7, 13×13, 19×19	1.33e-15, 1.89e-15, 3.77e-15, 5.55e-15

Figure 8. Effect of α at σ = 1.4 (128×128 crop).

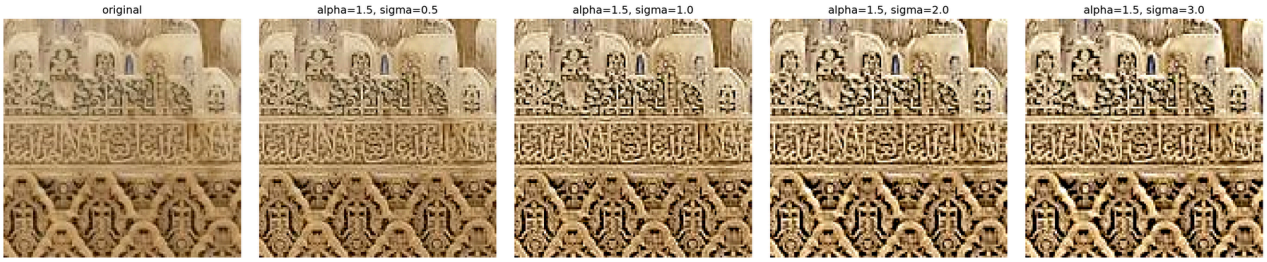

Figure 9. Effect of σ at α = 1.5 (128×128 crop).

The chosen setting is α = 1.5 and σ = 1.4. It separates the blurred stucco pattern clearly while the dark grooves do not yet turn black.

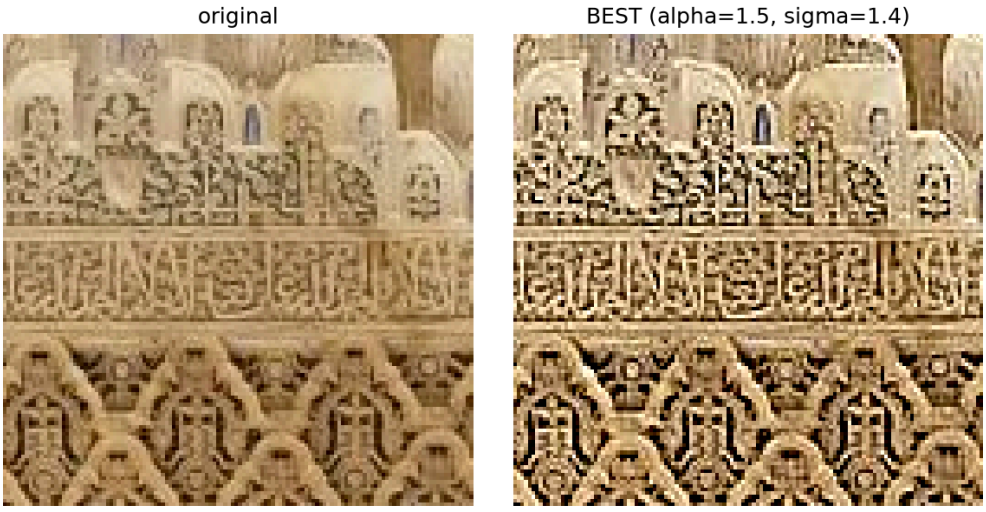

Figure 10. Original and chosen result (α = 1.5, σ = 1.4), crop.

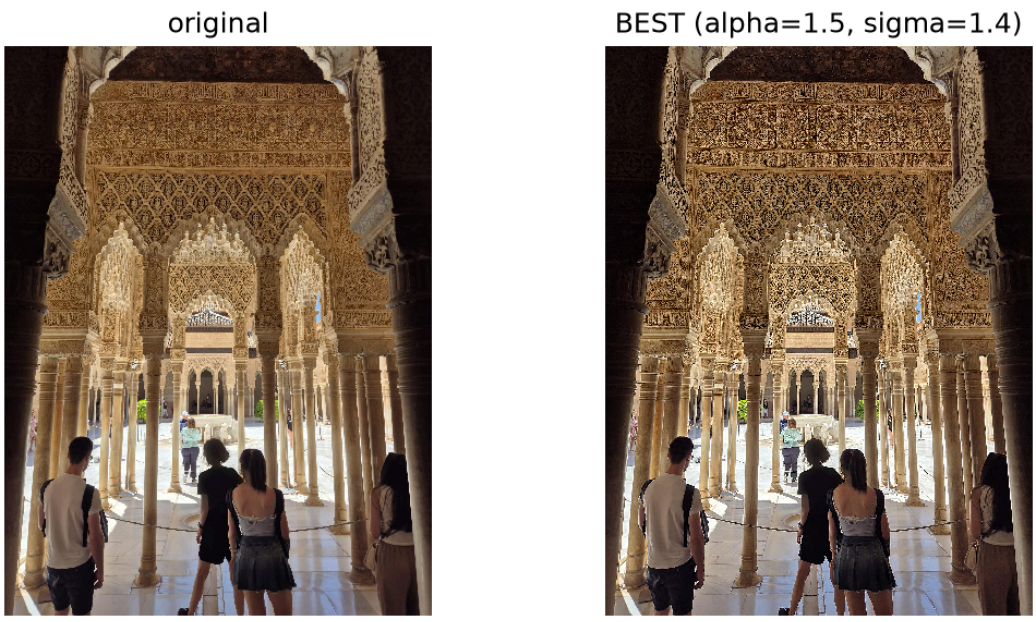

Figure 11. Original and chosen result (α = 1.5, σ = 1.4), full image.

Question. What are the respective effects of α and σ on the sharpening result?

Answer. α controls the strength, it scales the high frequency component that is added back. At α = 1 the carving becomes darker and the ridges brighter. As α grows to 2~4 this contrast becomes extreme. And even the artifacts are boosted also. However, σ controls the scale which decides what counts as detail. At σ = 0.5 only pixel-scale detail is boosted and the change is subtle. σ = 1–1.4 sharpens the fine ornament. At σ = 2–3 larger structures are boosted. The outlines around the ornament become thicker and darker, the overall local contrast rises, and halos become wider.

Part 2. Contrast Enhancement: Histogram Equalization

Implementation. The histogram h(r_k) of the 256 intensity levels is computed with `np.bincount`, the cumulative histogram with `np.cumsum`, and s_k = c(r_k)/MN. The mapping T(r_k) = round(255·s_k) is applied as a lookup table. For the color image, RGB is converted to YCbCr (BT.601 full range, implemented with its conversion matrix), only Y is equalized, Cb and Cr are kept, and the result is converted back to RGB. (scripts/Image_Enhancement/part2_2.py)

Details.

- Y is quantized to 8 bits (0–255) before the histogram, so the 256 levels r_k match the equation. s_k lies in [0, 1] and is scaled by L − 1 = 255 to map back to intensities.
- The mapping is a cumulative sum, so it is monotonic: the order of intensities is preserved. This was checked in the log (range 0–255, monotonic = True).
- Equalizing R, G and B separately would apply a different mapping to each channel and shift the colors. Equalizing only Y changes brightness while keeping the chroma Cb and Cr.
- After the inverse conversion the RGB values are clipped to [0, 1].

Output.

Figure 12. Original and histogram-equalized image (Y channel of YCbCr).

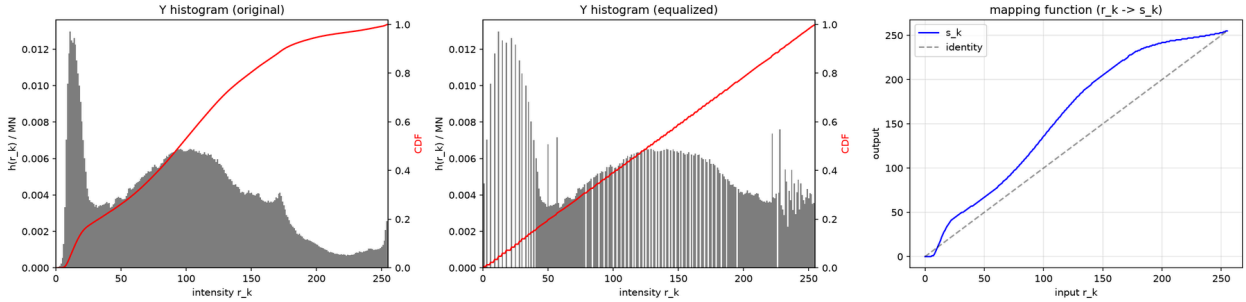

Figure 13. Y histogram with CDF (red) before and after equalization, and the mapping function s_k.

Histogram. The original Y histogram has a tall peak at about 10–20 (dark pillars), a broad mode around 100, and a spike at 255 (bright courtyard). After equalization the CDF is close to a straight line, as intended. The histogram becomes comb shaped with empty bins.

Mapping. The mapping curve lies above the identity everywhere, so the whole image becomes brighter.

Failure case. This image is a failure case of global equalization. The output range is allotted in proportion to the number of pixels, so the many dark and mid-tone pixels receive most of it, while the relatively few bright pixels of the courtyard and floor are squeezed into the top of the range. The courtyard therefore merges into white, mid-tones look muddy, and colors in the brightened regions look faded because Y increases while Cb and Cr stay the same. The steep slope in the darkest range also amplifies noise there. It shifts too much and it makes the image muddy.

3. HDRI and Tone Mapping

All three exposure sets in the HDRI folder are processed. A shared module, `hdr_common.py`, loads each set with its exposure times and defines the weighting functions.

Part 1. Radiometric Calibration: Linearize Bracketed Images

Implementation. N = 1000 pixel positions are sampled at random and read from every exposure. For each channel the system $\|Av - b\|^2$ is built and solved with `np.linalg.lstsq`, giving g and log E_i. The default is $\lambda = 50$, and $\lambda \in \{1, 10, 50, 100, 1000\}$ is compared. The images are linearized as I_lin = exp(g(I)) with a lookup table. The estimated g is saved to `crf.npz` for Part 2.
(scripts/HDRI_and_Tone_Mapping/part3_1.py: `build_system`, `solve_crf`, `linearize`)

Details.

1. **Unknown vector (1.1).** Columns 0–255 of A correspond to g(0)…g(255) and columns 256…256+N−1 to log E_1…log E_N.
2. **Data equations (1.2).** The N·P rows are filled at once: `np.repeat` and `np.tile` give the pixel index i and exposure index j of each row, +w_ij is written in column I_ij, −w_ij in column 256 + i, and b = w_ij·log Δt_j.
3. **Scale constraint (1.3).** One row with A[NP, 128] = 1 and b = 0 fixes g(128) = 0.
4. **Smoothness (1.4).** For z = 1…254 one row holds $\sqrt{\lambda}\cdot w(z/255)\cdot[1, -2, 1]$ at columns z − 1, z, z + 1, with b = 0.
5. **Solution (1.5).** A is (NP + 255) × (256 + N): 9255 × 1256 for image_1, 6255 × 1256 for office and 3255 × 1256 for tower. This is small enough for a dense matrix, so no sparse representation is needed. R, G and B are solved separately.
6. **Weighting.** The tent w(z) = min(z, 1 − z) is used without the Z_min/Z_max cut off for the CRF. With the cut-off of Part 2 (0.05/0.95), every data and smoothness equation for z below about 13 or above about 242 would have weight 0 and g would be undetermined there. Part 2 applies the cut-off when merging.
7. **Display.** The linearized images in the figure are subsampled. The lookup table works pixel by pixel, so subsampling before or after it gives the same values.

Output.

λ	weighted data residual RMS (R, G, B)	mean $\lvert\nabla^2 g\rvert$
1	0.0305, 0.0281, 0.0257	0.0494
10	0.0309, 0.0285, 0.0261	0.0163
50	0.0310, 0.0287, 0.0263	0.0064
100	0.0311, 0.0288, 0.0263	0.0044
1000	0.0312, 0.0289, 0.0264	0.0016

Results from image_1. The same trend holds for office and tower.

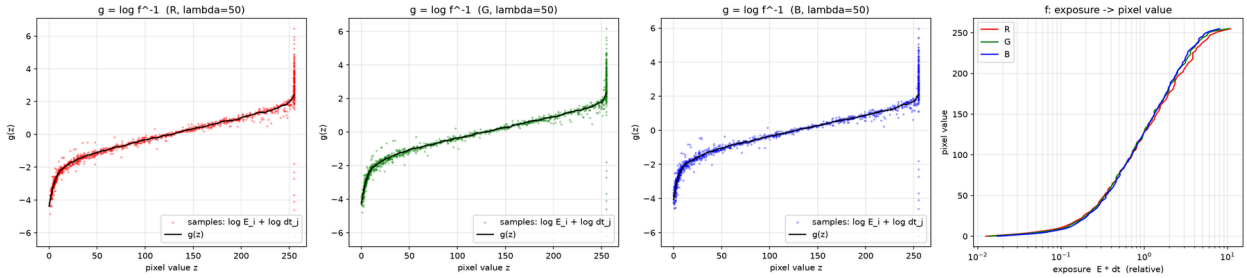

Figure 14. image_1: g = log f^{-1} for R, G, B (λ = 50) with sample points, and f (exposure to pixel value).

CRF. g is steep near z = 0, almost linear in the middle and rises sharply near 255, which is the typical shape of a camera response. In the middle range the sample points log E_i + log Δt_j lie close to the curve. The points at z = 255 spread vertically: they are saturated, get weight 0 and carry no information. The curves of R, G and B are nearly identical.

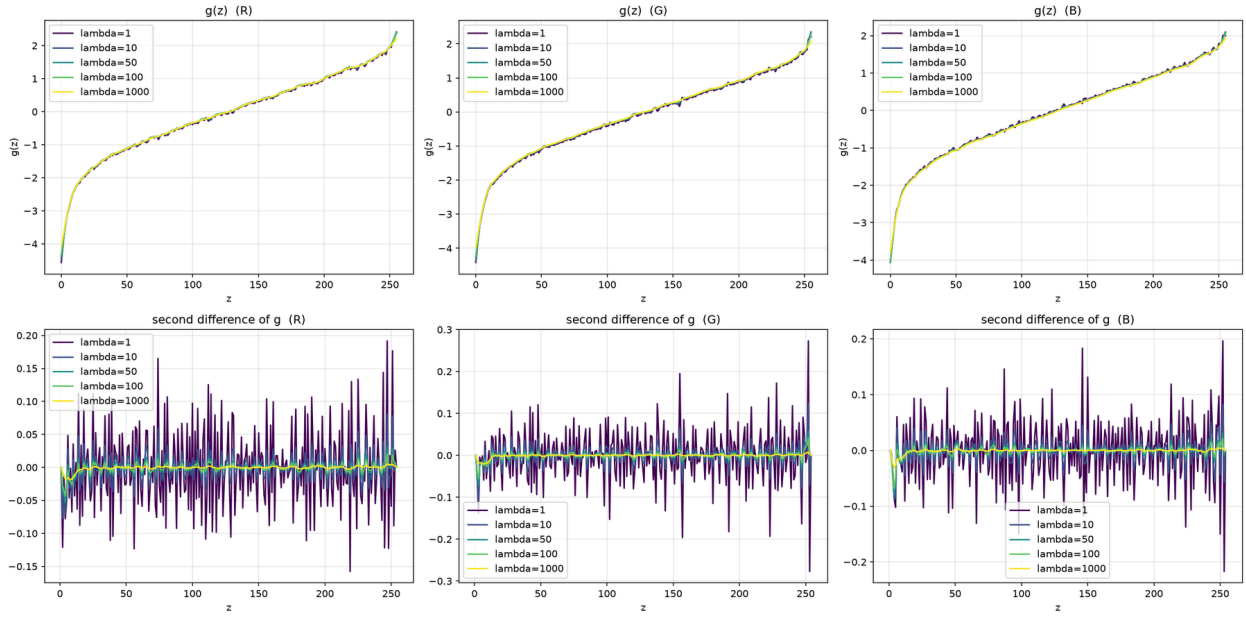

Figure 15. image_1: g for λ = 1, 10, 50, 100, 1000 (top) and its second difference (bottom).

Effect of λ. The overall shape of g does not change with λ. λ = 1 leaves small wiggles (second difference up to about ±0.2–0.3), and from λ = 100 the second difference is nearly flat. The data residual grows only from 0.0305 to 0.0312 (R) while mean $|\nabla^2 g|$ drops from 0.049 to 0.0016, so smoothing costs almost nothing in fit here. The visible exception is the dark end, where the true curve bends most: at z = 0 the λ = 1000 curve starts higher (about −4.0 against −4.5 for λ = 1 in R), i.e. heavy smoothing flattens the strongly curved part. A value in the recommended range (10–100) is therefore a good compromise, and λ = 50 is used.

Figure 16. image_1: input exposures (top), linearized exp(g(I)) (middle), and exp(g(I))/Δt (bottom). Display gamma 2.2.

Linearization. After dividing by Δt, the six exposures from 1/500 s to 1/40 s look alike although their inputs differ by more than 3 stops, which shows that the images are now proportional to radiance. In 1/15–1/6 s the window region is saturated: its linearized value is the clipped maximum, so dividing by Δt gives a flat, too dark area.

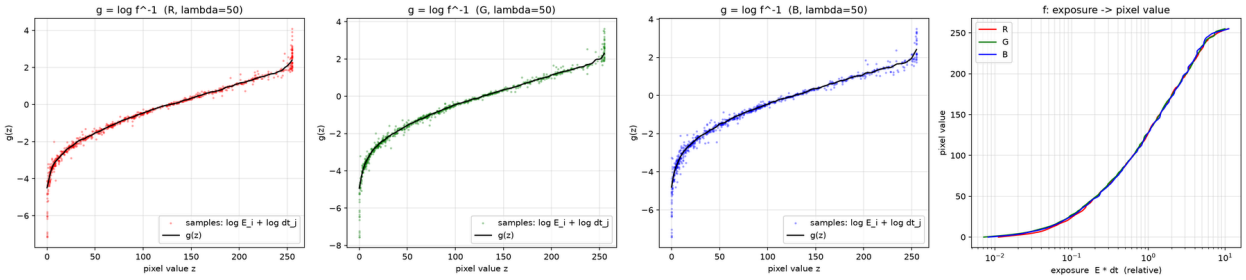

Figure 17. office: g and f (λ = 50).

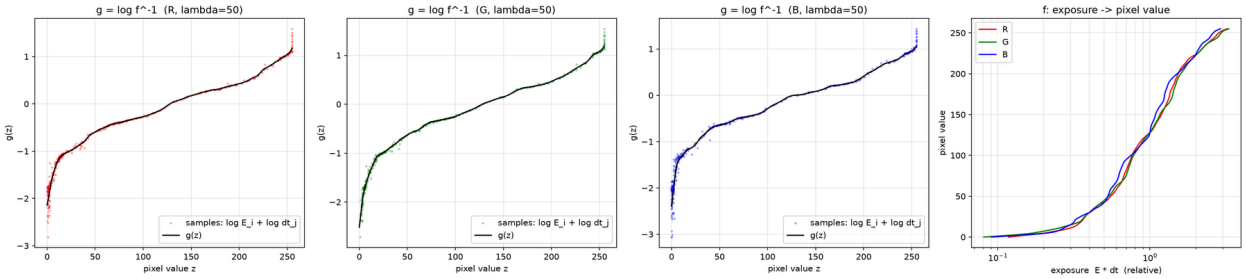

Figure 18. tower: g and f (λ = 50).

Other sets. office gives the same stable shape. tower is the least reliable: with only 3 exposures each pixel contributes 3 equations, g shows bumps near z ≈ 50, 100, 125 and 175, and the f curves of R, G and B differ visibly. Its small residual reflects the small number of constraints rather than accuracy. Its exposure ratios are also only assumed (2^(k−1)). So, if the true ratios differ, the whole g is distorted.

Part 2. Merge Exposure Brackets into HDR Image

Implementation. For each pixel and channel, the weighted average of log(I_lin + ε) − log Δt_j over all exposures is computed and exponentiated, with ε = 1e-6 and the g from Part 1. The three weighting functions (uniform, tent, Gaussian, Z_min = 0.05, Z_max = 0.95) are evaluated in the same pass. The tent result is saved as Radiance `.hdr` with `cv2.imwrite` and read back to check it. (scripts/HDRI_and_Tone_Mapping/part3_2.py: `merge_hdr`)

Details.

- **Lookup tables.** The weight and log(exp(g(z)) + ε) depend only on the 8-bit value z, so both are 256-entry tables. Each exposure is read once and accumulated into numerator and denominator arrays of shape (3, H, W) in float32. This keeps the 5472×3648 image_1 set manageable at full resolution.
- **Zero weight sum.** A pixel whose value is outside [Z_min, Z_max] in every exposure (saturated in all of them or dark in all of them) has Σw = 0. For such pixels the exposure whose z is closest to 0.5 is used instead. The shortest exposure for highlights that clip everywhere, the longest for very dark pixels. This affects 1.42 % of the pixels in image_1, 0.08 % in office and 1.63 % in tower. The three weights share the same support, so the fraction is identical for all of them.
- **Comparison of weights.** Besides previews, the difference |log2 E_k − log2 E_tent| (in stops) is shown as a map, together with crops where it is largest (A) and in a dark region (B). All previews use the same simple global tone curve so that they are comparable.

Output.

Set	Σw = 0 (fallback)	Dynamic range (0.1–99.9 %)	uniform − tent: mean / 99.9 %	Gaussian − tent: mean / 99.9 %
image_1	1.42 %	10.8 stops	0.040 / 0.32 stops	0.018 / 0.16 stops
office	0.08 %	10.1 stops	0.027 / 0.29 stops	0.012 / 0.14 stops
tower	1.63 %	5.5 stops	0.005 / 0.07 stops	0.002 / 0.03 stops

Figure 19. image_1: weighting functions and HDR previews for each weight.

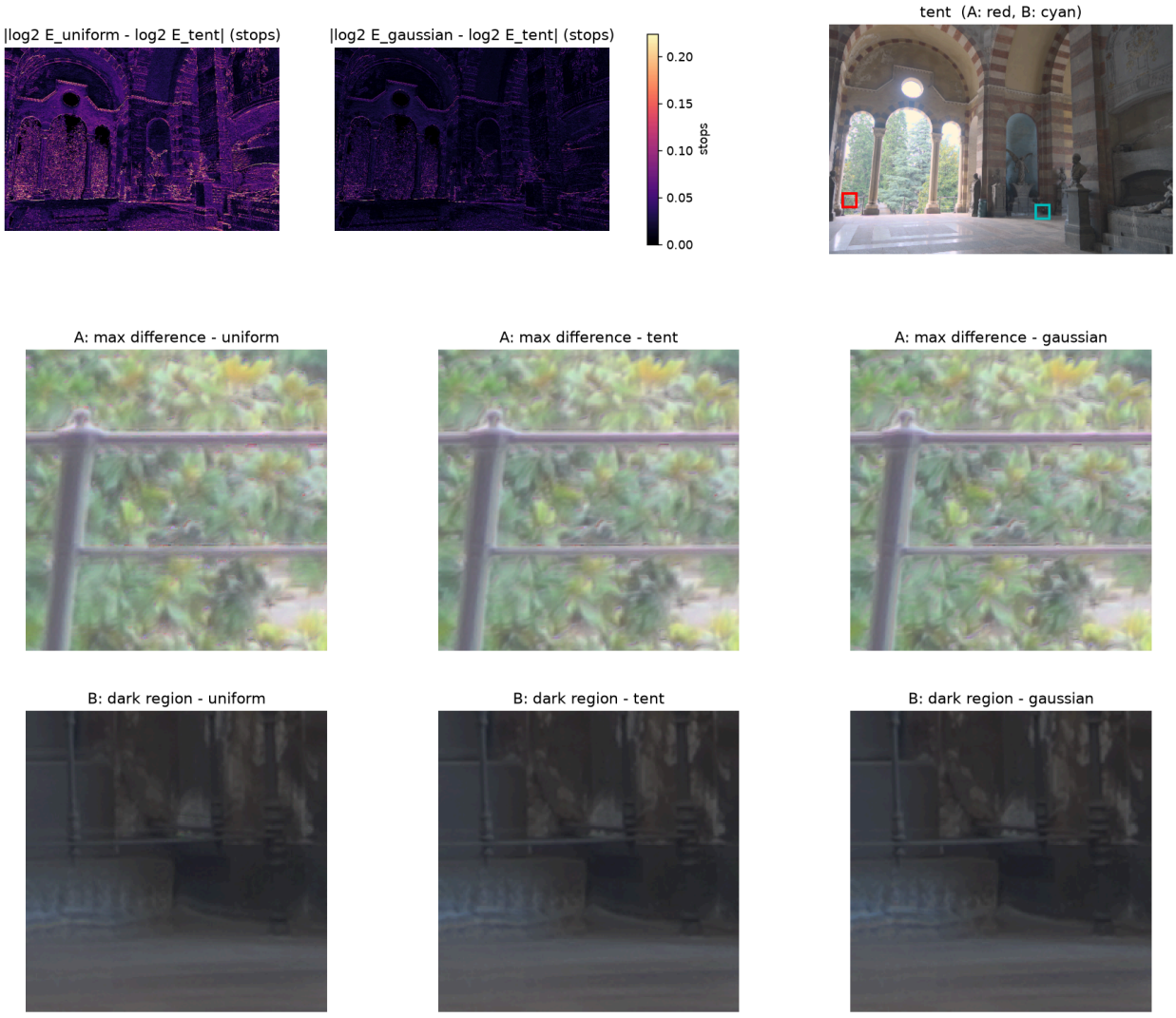

Figure 20. image_1: difference to the tent result in stops, and crops A (largest difference) and B (dark region).

Comparison. At preview size the three results are almost indistinguishable. The mean difference is 0.04 stops (about 3 % in radiance) for uniform and 0.018 stops for Gaussian, compared with tent. The weights only change how the radiance estimates of the individual exposures are averaged. When the CRF is accurate and the exposures agree, the average is nearly independent of the weights, so the differences concentrate on edges and textured regions, while flat regions show almost no difference.

Ghosting. In image_1 the largest differences are in the foliage behind the arches (crop A). The leaves move between exposures taken over 1/500–1/6 s, so the exposures do not agree there and the result depends on the weights. The merged image shows ghosting in this region with every weight. Weighting cannot remove motion between exposures. office and tower show no such region since it is static scenes, and tower has only 3 exposures. And their differences are even smaller.

Result. No weight is clearly more pleasing visually. I selected tent as a method. Unlike uniform, it fades an exposure in and out smoothly at Z_min/Z_max instead of switching it on and off, it is the weight of Debevec and Malik and the one used for the CRF, and the Gaussian result is within 0.02 stops of it.

Part 3. Tone Mapping

Implementation. The luminance L_w = 0.2126R + 0.7152G + 0.0722B is scaled by the log average luminance with key a = 0.18. Both global operators are applied to the scaled luminance L, and the color is restored by scaling RGB with L_d/L_w. The result is clipped to [0, 1], encoded with gamma 1/2.2 and stored in 8 bits. (scripts/HDRI_and_Tone_Mapping/part3_3.py)

Details.

- B ∈ {0.1, 0.25, 0.5, 1.0} and β ∈ {1, 10, 100, 1000} are compared. The settings saved at full resolution are B = 1.0 and β = 10.
- Both operators are global: the curve depends only on $\bar{L}$ and max(L), which are computed on the full resolution image. The comparison figures apply the same curve to a subsampled image.
- Scaling RGB by L_d/L_w keeps the ratios between channels, so hue is preserved. Channels that exceed 1 after scaling are clipped.
- (c) The provided `MMSPG_0026.hdr` (1920×1080) is processed with the same code. Only the HDR file is provided, so its result is compared with the scaled image without tone mapping instead of input exposures.

Output.

Image	log-average $\bar{L}$	max(L) (scaled)
image_1	44.08	20.86
office	0.860	6.41
tower	0.321	1.82
MMSPG_0026	79.32	4.81

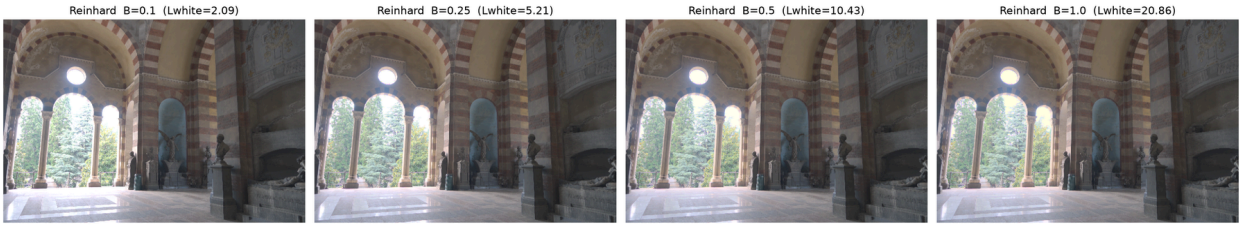

Figure 21. image_1: Reinhard with B = 0.1, 0.25, 0.5, 1.0.

Figure 22. image_1: Logarithmic with β = 1, 10, 100, 1000.

Figure 23. image_1: input exposures (1/500, 1/60, 1/6 s), scaled image without tone mapping, Reinhard (B = 1.0) and Logarithmic (β = 10).

Figure 24. MMSPG_0026 (c): Reinhard with B = 0.1, 0.25, 0.5, 1.0.

Figure 25. MMSPG_0026 (c): Logarithmic with β = 1, 10, 100, 1000.

Figure 26. MMSPG_0026 (c): without tone mapping, Reinhard (B = 1.0) and Logarithmic (β = 10).

Figure 27. office: input exposures, without tone mapping, Reinhard (B = 1.0) and Logarithmic (β = 10).

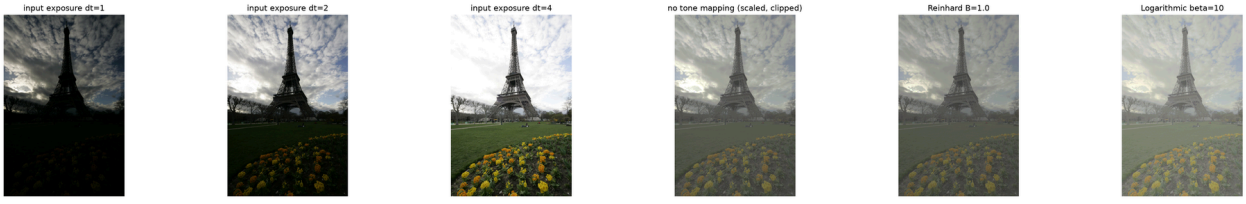

Figure 28. tower: input exposures, without tone mapping, Reinhard (B = 1.0) and Logarithmic (β = 10).

Discussion.

Parameter B. L_white is the luminance that is mapped to white, and every L above it gives L_d > 1 and is clipped. With B = 0.1 (L_white = 2.09 for image_1, 0.48 for MMSPG) the windows burn out. B = 1.0 sets L_white = max(L), so nothing is clipped, at the cost of a flatter image. B = 0.25–0.5 lies in between.

Parameter β. β = 1 gives a dark, contrasty image. β = 100 and 1000 lift the shadows and mid-tones so much that black turns gray, and the image becomes hazy and desaturated for all four inputs. β = 10 is the most balanced.

Reinhard vs. logarithmic. To compare the two operators fairly, both should keep the highlights: B = 1.0 against β = 10. For MMSPG (L_max = 4.81) the two curves give:

Scaled L	Reinhard (B = 1.0)	Logarithmic (β = 10)
0.18 (mid-tone)	0.154	0.265
2.0	0.724	0.782
3.0	0.847	0.882
4.81 (max)	1.0	1.0

At B = 1.0 both keep the sky and clouds behind the window panes, which are clipped to white without tone mapping, and their highlights are within a few percent of each other. The difference is in the dark and mid-tones, which the logarithmic curve maps about 1.7 times brighter. With B = 0.5, Reinhard clipped every L above 2.4, which is why the window looked better in the logarithmic result before B was raised.

Result. I think that logarithmic operator (β = 10) is better on MMSPG. It keeps the view outside the window and shows the plant detail and colors in the shaded interior better, while Reinhard renders the flowers slightly darker.

And Reinhard keeps more mid-tone contrast, and on image_1 the two results look similar. tower is a weak case for both, so Reinhard is almost identical to the image without tone mapping, and the result looks flat and gray, consistent with its 5.5 stop HDR.

Comparison with the input exposures. No single exposure shows both the bright and the dark parts of the scene. In image_1 the 1/500 s exposure shows the trees behind the arches but leaves the interior black, and the 1/6 s exposure shows the interior but turns the arches white. The tone mapped images show both at once. The image scaled without compression clips the highlights (the MMSPG window becomes white), which shows that the range has to be compressed and not only scaled.

Question. What information is lost or compressed during tone mapping?

Answer. Tone mapping fits about 10~11 stops into one 8 bit range, so highlights are clipped or strongly compressed and dark detail is lost or amplified together with noise, depending on the curve. Absolute radiance and true brightness ratios are lost, nearby values merge after 8-bit quantization, and a global curve cannot adapt to local regions.

Use of AI Tools

Generative AI (Claude, GPT) was used in preparing this report: translating and formatting the text. And also used for making visualization codes.